FRED TALKS

26th May 2026

CONTENTS

Building agents that reach production systems with MCP

CLAUDE.COM · 23 APR 2026 · SOURCE

Agents are only as useful as the systems they can reach. Teams tend to converge on three approaches for connecting them to external systems—direct API calls, CLIs, and MCP. This post lays out where each fits, why production agents tend to land on MCP, and the patterns for building those integrations effectively.

Connecting agents to external systems

We generally see three paths for connecting agents to external systems: direct API calls, CLIs, and MCP. Each makes sense somewhere, depending on what you're building. The key distinction is whether there's a common layer between agents and services, and how far that layer reaches.

Direct API calls

The agent calls your API directly—either by writing code that issues HTTP requests inside a code-execution sandbox, or through a generic function-calling tool. This is where most teams start, and it works fine for one agent talking to one service, or a small number of integrations that don't need to be reused across agent platforms.

The challenges start to hit at scale. With no common layer between agents and services, each agent–service pair becomes a bespoke integration with its own auth handling, tool descriptions, and edge cases—the M×N integration problem.

Command-line interface (CLI)

The agent runs your command-line tool in a shell. This is fast, lightweight, and leans on pre-existing tooling. It works great for local environments and sandboxed containers—anywhere there's a filesystem and a shell. This provides a common layer, but it’s thin.

CLIs hit hard limits reaching mobile, web, or cloud-hosted platforms that don't expose a container, and auth is handled by the CLI's own mechanism—usually a credential file on disk. This is best suited to quick, permissive integrations in local environments.

Model Context Protocol (MCP)

MCP provides the common layer as a protocol. The agent connects to a server that exposes your system's capabilities, with auth, discovery, and rich semantics standardized. One remote server reaches any compatible client (Claude, ChatGPT, Cursor, VS Code, and more), in any deployment environment.

It requires a little bit more upfront investment. The return is that the integration is portable, and provides the semantics needed for a feature-rich agent integration.

Production agents run in the cloud

Production agents increasingly run in the cloud, so they can scale and operate continuously. The systems they need to reach are cloud-hosted too: where your data lives, work is tracked, and your infrastructure runs. Often these systems are remote and behind auth, where MCP provides the common layer.

We're already seeing this in adoption. The MCP SDKs recently surpassed 300 million downloads a month, up from 100 million at the start of the year, with strong adoption across enterprises and popular agentic platforms. Millions of people use MCP with Claude every day, and the protocol underpins much of what we've shipped recently, including Claude Cowork, Claude Managed Agents, and channels in Claude Code.

As MCP continues to support production agentic systems, we're sharing patterns for building these integrations well: from building advanced servers to context-efficient clients, and where skills complement the protocol.

Building effective MCP servers

We have over 200 MCP servers in our directory, used by millions of people every day. From working closely with enterprises and developers building on the protocol, we've spotted a handful of design patterns that determine how reliably agents can use a server.

Build remote servers for maximum reach

A remote server is what gives you distribution—it's the only configuration that runs across web, mobile, and cloud-hosted agents, and it's what every major client is optimized to consume. Build remote servers so agents can use your system wherever they run.

Group tools around intent, not endpoints

Fewer, well-described tools consistently outperform exhaustive API mirrors. Don't wrap your API into an MCP server one-to-one—group tools around intent, so the agent can accomplish a task in a couple of calls instead of stitching many primitives together. A single create_issue_from_thread tool beats get_thread + parse_messages + create_issue + link_attachment. See writing effective tools for agents to learn more about the full pattern.

Design for code orchestration when your surface is large

If your service requires hundreds of distinct operations, such as Cloudflare, AWS, or Kubernetes, an intent-grouped toolset likely won't cover it. Instead, expose a thin tool surface that accepts code: the agent writes a short script, your server runs it in a sandbox against your API, and only the result returns. Cloudflare's MCP server is the reference example—two tools (search and execute) cover ~2,500 endpoints in roughly 1K tokens.

Ship rich semantics where they help

MCP Apps is the first official protocol extension and lets a tool return an interactive interface, such as a chart, form, or dashboard, all rendered inline in the chat interface. Servers that ship MCP apps tend to see meaningfully higher adoption and retention than those that return text alone. Use it to put your product's UI in front of agents or end-users at the moment it matters—the extension is supported in Claude.ai, Claude Cowork, and many other top AI tools.

Elicitation lets your server pause mid-tool call to ask the user for input. Form mode sends a simple schema and the client renders a native form—use it to request a missing parameter, confirm a destructive action, or disambiguate options. URL mode hands the user to a browser—use it to complete downstream OAuth, take a payment, or collect any credential that should never transit the MCP client. Both keep the user in the flow instead of sending them to a settings page. Form mode is supported broadly; URL mode is supported in Claude Code, with more clients in progress.

Lean on standardized auth

Standardized auth makes MCP practical for cloud-hosted agents. If your server requires OAuth, the latest MCP spec supports CIMD (Client ID Metadata Documents) for client registration—it gives users a fast first-time auth flow and far fewer surprise re-auth prompts. This is our recommended approach for auth, the capability is supported in MCP SDKs, Claude.ai, and Claude Code, and is being broadly adopted across the industry.

Once a user has authorized, the next question is how a cloud-hosted agent holds and reuses those tokens at runtime. Vaults in Claude Managed Agents covers this: register a user's OAuth tokens once, reference the vault by ID at session creation, and the platform injects the right credentials into each MCP connection and refreshes them on your behalf—no secret store to build, no tokens to pass around per call.

Making MCP clients more context-efficient

MCP standardizes how AI agents (*clients*) connect to and work with tools and data sources they need (*servers*). The server securely exposes a range of capabilities, while the client orchestrates them and manages context. If you're building an MCP client, make it context-efficient with patterns for progressive disclosure.

Load tool definitions on demand with tool search

Tool search defers loading all tools into context, rather than loading them upfront. This allows the agent to search the catalog at runtime, pulling in the relevant tools when needed. In our testing, tool search tends to cut tool-definition tokens by 85%+ while maintaining high selection accuracy.

Reducing context usage with tool search. Source: advanced tool use

Process tool results in code with programmatic tool calling

Programmatic tool calling processes tool results in a code-execution sandbox, rather than returning them raw to the model. This lets the agent loop, filter, and aggregate across calls in code, with only the final output reaching context. In our testing, this reduces token usage by roughly 37% on complex multi-step workflows.

Together, these patterns compose naturally across multiple servers: leaner context, fewer round-trips, faster responses. See *advanced tool use* for the full breakdown.

Pairing MCP servers with skills

Skills and MCP are complementary. MCP gives an agent access to tools and data from external systems, while skills teach an agent the procedural knowledge of *how* to use those tools to accomplish real work. The most capable agents use both, and skills make MCP servers scale beyond a handful of connections. There are two general patterns for combining them:

Bundle skills and MCP servers as a plugin

Plugins for Claude are a useful abstraction that allow developers to bundle skills, MCP servers, hooks, LSP servers, and specialized subagents in one easily-consumable distribution method. Using this approach is the best way to unify multiple context providers with minimal friction.

Combining MCP servers with skills allows Claude to act more like a domain-specialist. Grab your tools via MCP, and give Claude the skills to orchestrate workflows end-to-end. See our data plugin for Cowork as an example, which consists of 10 skills and 8 MCP servers for apps like Snowflake, Databricks, BigQuery, Hex and more.

Combining skills with MCP. Source: Extending Claude's capabilities with skills and MCP servers

Distribute skills from an MCP server

It's increasingly common for providers to publish a skill alongside their MCP server, so the agent gets both the raw capabilities and an opinionated playbook for using them well. Canva, Notion, Sentry, and more do this today in Claude, listing the skill next to their connector in our web directory.

To make that pairing portable across every client, the MCP community is actively working on an extension for delivering skills directly from servers. This way the client inherits the relevant expertise automatically, versioned with the API it depends on. We expect this pattern to see broad adoption as the extension stabilizes.

The compounding layer

We opened with three paths for connecting agents to external systems. In practice, mature integrations will ship all three: the API as the foundation, a CLI for local-first environments, and MCP for cloud-based agents.

As production agents move to the cloud, MCP becomes the critical layer, and it's the one that compounds. Today, a remote server reaches every compatible client across any deployment environment, with auth, interactivity, and rich semantics handled by the protocol. As more clients adopt the spec and more extensions land in it, that same server gets more capable without you shipping anything new.

When building an integration, if your goal is to have production agents in the cloud reach your system, build an MCP server and make it excellent using the patterns above. Every integration built on MCP strengthens the ecosystem: fewer edge cases to solve alone, fewer bespoke integrations to maintain.

Acknowledgements

Thanks to Den Delimarsky, David Soria Parra, Henry Shi, Felix Rieseberg, Conor Kelly, Molly Vorwerck, Andy Schumeister, Kevin Garcia, Amie Rotherham, Matt Samuels, Angela Jiang, Katelyn Lesse, AJ Rebeiro and Jess Yan for their contributions to this blog.

Evolution and Scale of Uber's Delivery Search Platform

DIVYA NAGAR · 05 DEC 2025 · SOURCE

Featured image for Evolution and Scale of Uber's Delivery Search Platform

Introduction

Search is a primary discovery funnel for Uber Eats: a large share of orders start with people typing into the search bar to find stores, dishes, and grocery items. Strong search directly translates into higher conversion, better basket quality, and faster time-to-order—especially for long-tail queries, new or seasonal items, and multilingual markets. When search misses intent, people bounce or fall back to browsing. When it understands intent, they find what they want in seconds.

Traditional search stacks begin with lexical matching, which is fast and effective when queries exactly match document text. But real queries are challenging—synonyms (“soda” versus “soft drink”), typos (“mozzarela”), shorthand (“gf pizza”), language mix (“pan” meaning bread in Spanish, but a container for cooking in English), and context (“apple” the fruit vs the company). Lexical methods see strings, not meaning, and aren’t suitable for such queries, producing bad search results.

Semantic search shifts from matching words to matching meaning. It encodes queries and documents into vectors in the same space, so semantically similar things are close—even without keyword overlap. When properly used, it delivers a search experience that better captures someone’s intent across verticals (stores, dishes, items) and languages.

Establishing semantic search at scale for industry applications is more than training a model, but instead a whole tech stack including model deployment, ANN (Approximate Nearest Neighbor) index serving at scale, monitoring, version control, and so on. This blog walks you through how Uber Eats built this system, reviewing challenges in this process and the solutions we took.

Architecture

The general semantic search model architecture is shown in Figure 1. We adopted the classic two-tower structure so that the query and document embedding calculation can be decoupled. The query embeddings are calculated via an online service in real time, while the document embeddings are calculated in a batch manner via offline scheduled services. When training the model, we used the MRL (Matryoshka Representation Learning)-based infoNCE loss function so that a single model could output various embedding dimensions, and downstream tasks could choose the optimal embedding dimension according to their own size versus quality tradeoffs.

The latest LLMs (QWEN®) are used as the backbone embedding layer inside two towers to leverage its world knowledge and cross-lingual abilities. We further finetuned them using Uber Eats in house data to adapt the embedding space to our application scenarios. Currently, this single embedding model supports the content embedding task for all Uber Eats verticals and markets.

Figure 1: Model training architecture.

Model Training and Inference

Our model is a two-tower deep neural network built on a Qwen backbone. We use PyTorch™ with Hugging Face Transformers® for model development and Ray™ for distributed training orchestration. For large-scale training, we use PyTorch's DDP (Distributed Data Parallel) together with DeepSpeed® (ZeRO-3), which shards the optimizer, gradients, and model parameters to handle the immense size of the Qwen model. This setup, combined with mixed-precision training and gradient accumulation, allows us to train on hundreds of millions of data points efficiently. Each successful training run generates versioned artifacts—a unique ID for the query and document models (*query_model_id*, *doc_model_id*), and a shared team-specific ID called two-tower-embedding-id (*tte_id*)—which are meticulously tracked for reproducibility and deployment.

Given the scale of our document corpus, offline inference is critical. To manage costs, we embed documents at the feature level (for example, store or item) and then join these embeddings back to the full catalog of billions of candidates before indexing. The resulting embeddings are stored in stable feature store tables, keyed by entity IDs, making them readily available for both retrieval and ranking models. These embeddings are used to build our search index, which are supported by HNSW graphs and store both non-quantized (float32) and quantized (int8) vector representations to optimize for various latency and storage budgets.

Figure 2: End-to-end training inference workflow.

Development and Deployment Challenges

We tackled key challenges in scaling our semantic search system. For example, we had to balance retrieval accuracy with infrastructure and compute costs through careful tuning of ANN parameters, quantization, and embedding dimensions.

To achieve productionization and reliability, we had to ensure safe, automated, and rollback-capable index deployments with strong validation, versioning, and real-time consistency checks in production.

Quality and Cost

We use Apache Lucene® Plus as our primary indexing and retrieval system. To manage the scale and diverse nature of the data, we maintain separate indices for each vertical—OFD (restaurants) and GR (grocery/retail).

Storing and serving large-scale indexes can be expensive, in terms of infrastructure costs and query latency. To optimize this tradeoff while maintaining strong retrieval quality, we explored several mechanisms to balance cost, latency, and performance. In particular, we ran controlled experiments varying three key factors:

- **Search parameter K**: controls the number of nearest neighbors retrieved per shard in ANN search. By carefully choosing k per shard—guided by data distribution statistics—you can maintain the overall retrieval cohort size and preserve result quality.
- **Quantization strategy**: comparing non-quantized float32 embeddings against compact int7 SQ (scalar-quantized) representations.
- **Embedding dimension**: using different Matryoshka-cut representations generated via MRL (Matryoshka Representation Learning), such as *emb_256* and *emb_512*.

Here's how we designed search parameters and quantization:

Image

Table 1: Impact of shard-level k and quantization on recall and cost savings.

We also used MRL (Matryoshka representation learning) to reduce the dimension of the embeddings to optimize the latency and storage costs further. MRL solves this by training a single model whose embeddings can be cut at different lengths. Shorter versions of the embedding capture coarse information quickly, while longer versions add finer details for higher accuracy. This gives us the flexibility to trade off speed versus quality at inference time, without retraining separate models.

In our application, we chose the following vector dimensions with equal weights: [128, 256, 512, 768, 1024, 1280, 1536]. In our offline evaluation results, we observed that the MRL provides strong performance in dimension reduction.

Image

Table 2: Dimension reduction performance difference from MRL.

Lowering shard-level *k* from 1,200 to around 200 yielded a 34% latency reduction and 17% CPU savings in our experiments, with negligible impact on recall. On top of that, applying scalar quantization (int7) further cut compute costs, reducing latency by more than half compared to fp32 while maintaining recall above 0.95. MRL added another layer of efficiency

by enabling us to serve smaller embedding cuts (256 dimensions) with almost no loss (< 0.3% for English and Spanish) in retrieval quality compared to full-size embeddings, while also reducing storage costs by nearly 50%. Together, these optimizations significantly lowered the cost and latency of serving large-scale indexes without any significant compromising in retrieval quality.

Beyond quantization and k tuning, our vector index is paired with locale-aware lexical fields and boolean pre-filters—including *hexagon*, *city_id*, *doc_type*, and *fulfillment_types*—that run before the ANN search. These pre-filters dramatically shrink the candidate set up front, ensuring low-latency retrieval even against a multi-billion document corpus. Once the ANN search identifies the top-k candidates per vertical, we optionally apply a lightweight micro-re-ranking step, using a compact neural network, to further refine results before handing them off to downstream rankers.

Productionization and Reliability

Uber data changes continuously. To keep results fresh, our Delivery semantic search runs a scheduled end-to-end workflow that updates the embedding model and serving index in lockstep on a biweekly cadence. Each run trains or fetches the target model, generates document embeddings, and builds a new Lucene Plus index without changing the live read path.

To enable seamless model refreshes without disrupting live traffic—and to provide rollback flexibility when issues arise—we adopted a blue/green pattern at the index column level. Each index maintains two fixed columns, *embedding_blue* and *embedding_green*, with the active model version mapped to one of them via a lightweight configuration. During each refresh, a new index snapshot is built containing both columns: the active column is preserved as-is, while the inactive column is repopulated with fresh embeddings. This design allows us to safely switch between old and new versions when needed.

During each refresh, we rebuild a full index snapshot that contains both columns (*embedding_blue* and *embedding_green*). Even though only the inactive column is intended to change, the build re-materializes the active column as well, so a bug or bad input could silently corrupt what's currently serving. To protect production, we gate with automated validations that run before any deployment: they block the flow if inputs are incomplete, if the carried-forward (active) column diverges significantly from the current prod index, or if the newly refreshed column underperforms.

Concretely, our pipelines run three checks.

- **Completeness:** verify document counts (and key filtered subsets) in the index match the source tables used to compute embeddings—fail fast on drift or partial ingestion.
- **Backward-compatibility:** ensure the unchanged column is byte-for-byte equivalent between the pre-refresh and post-refresh index (for example, at T1 compare *embedding_blue* across old and new builds if *embedding_blue* is currently serving. Any mismatch ends the run.
- **Correctness:** deploy the newly built index to a non-prod environment, replay real queries at a controlled rate, and compare recall against the current production index. These checks catch issues like data corruption during indexing, bad distance settings, or upstream feature regressions, ensuring the new index is at least as good as production before we make a switch.

We also considered maintaining two separate blue and green indexes instead of two columns within a single index. While that approach would simplify reliability—since we wouldn't need to touch the production index during a refresh—it'd significantly increase storage and maintenance costs. Given that trade-off, we built a mechanism that lets us safely operate with one index containing two columns.

Image

Figure 3: Model<>index deployment and serving flow.

Even with strong build-time and deploy-time checks, there's still a risk of human error or bugs. For example, a wrong model-to-column mapping or an index built against a different-than-expected model. To guard against this, we added serving-time reliability checks. Reliability of the production system was a significant challenge during the development process.

To make the system more reliable and prevent outages, we deploy the new index to production, then roll out the model gradually. The index stores the model ID in its metadata for both columns. On sampled requests, the service verifies that the model used to generate the query embedding matches the model ID recorded on the active index column. On any mismatch, we emit a compatibility-error metric and log full context. If errors remain non-zero over a short window, an alert fires and we automatically roll back the model deployment to the previous version (the index remains unchanged). This protects against version mix-ups and config mistakes without adding latency or extra hops to the read path.

Conclusion

We built a scalable, multilingual search system for Uber Eats that powers discovery across restaurants, grocery, and retail with speed and precision. Our approach brings together advanced language models and smart architecture choices—like Matryoshka embeddings and Lucene Plus integration—to balance quality, cost, and latency at a global scale.

What makes this system stand out is its production-first design: it automatically retrains, rebuilds, and refreshes search indexes every two weeks, ensuring that results stay fresh and reliable as data changes daily. With a robust blue/green deployment strategy, strong validation gates, and efficient index optimization, we can safely roll out new models without disrupting live traffic.

Overall, this platform demonstrates how thoughtful engineering—combining strong modeling, reliable infrastructure, and cost-aware optimization—can deliver a search experience that feels smarter, faster, and more intuitive for users around the world.

Stay up to date with the latest from Uber Engineering—follow us on LinkedIn for our newest blog posts and insights.

Divya Nagar

Divya Nagar

Divya Nagar is a Staff Software Engineer on the Feature Engine team at Uber, where she leads the development of the Vector Store, the data plane powering embedding use cases across both generative AI and CoreML applications.

Zheng Liu

Zheng Liu

Zheng Liu is a Staff Software Engineer on Uber Eats Search team, where he focuses on improving search experience using advanced deep learning and large language models. He's a major contributor to Uber Eats semantic search and generative ranking models.

Jiasen Xu

Jiasen Xu

Jiasen Xu is a Sr. Software Engineer on Uber's Search Platform team, where he leads the adoption of vector search technologies across the company. His work focuses on advancing large-scale similarity search through expertise in ANN algorithms, indexing libraries, and vector database optimization.

Bo Ling

Bo Ling

Bo Ling is a Sr. Staff Software Engineer on Uber's AI Platform team. He works on NLP, large language models, and recommendation systems. He's the lead engineer for embedding models and LLM on the team.

Haoyang Chen

Haoyang Chen

Haoyang Chen is a Sr. Staff Software Engineer on Uber Eats Search team, where he leads efforts in semantic search and generative recommendation to enhance the search experience on Uber Eats.

Posted by Divya Nagar, Zheng Liu, Jiasen Xu, Bo Ling, Haoyang Chen

Category:

Engineering

Data / ML

Uber AI

The lies I used to tell myself

CATE HALL · 23 JAN 2026 · SOURCE

1. What doesn’t kill you makes you stronger.

When I was 31, my husband of four years, who I loved more than my own life, left me completely out of the blue. We’d had an incredible, epic romance — drawn together like magnets, we married on our third date and spent the years that followed burrowing ever closer together. Maybe that was the problem — I can’t say for sure, because I never got a satisfying explanation out of him. One day I was the protagonist of a Princess Bride-worthy love story; 48 hours later, he moved out.

From the time we met, I had never contemplated the possibility of living without him, and I didn’t want to. I wanted to die for a long time — at least a couple of years. But as I slowly learned to sequester away thoughts of him and the emotions they brought up, I became convinced of a silver lining: The experience would make me impenetrable. If I survived it, I would know I could survive anything, and nothing would be able to hurt me again.

It took the better part of a decade and another deeply identity-shifting relationship, with the man who became my second husband, for me to see that I had it all wrong: Impenetrability means avoiding the parts of life that are most life-like. There’s a reason the visual metaphor for love is an arrow.

The near-fatal catastrophes of our lives don't always make us stronger. Surviving one hard thing does provide evidence that you can survive other hard things, which is easy to *mistake* for getting stronger. But in reality they often just make us brittle, make us dissociate and flee from the situations that remind us we can be broken.

2. When you know, you know.

Mario Gabriele of the Generalist had a great "50 Things" style post a few months ago that I found myself nodding along to almost the whole way through. The glaring exception was Number 33: "Love is easy. The difference between bad relationships and meeting your spouse is not a matter of degree, but category. Compatibility is unignorable and effortless."

I believed this completely, once. I've since come to believe that what's unignorable and effortless is chemistry. Chemistry can be powerful enough to turn your life sideways, but it's not the same thing as compatibility. Compatibility is litigated on the timescale of years and decades; it's not a momentary feeling but a description of what happens during an extended course of change. Do you grow toward one another, or do you grow apart? Does the relationship help you move closer to the person you aspire to be?

I "just knew" with my first husband. I wasn't sure with my second. Four years in, the signs have flipped.

3. If it were a good idea, someone would be doing it already.

There's a belief, especially common among wonky types, that you don't find $20 bills on the sidewalk — if an opportunity were real, someone would have seized it already. This sounds worldly and wise, but it's actually a form of defensive thinking. It assumes the current way is best, that everything worth doing is being done.

In my experience, this is just false. When I started playing poker, I discovered that physical tells were basically a wide-open field — despite the fact that most pros believed the topic was played out. If you think you see an opportunity others have missed, you should have a story for why they're missing it, and you should ask around to stress-test that story. But if no one can give you a satisfying reason, don't assume one exists. Sometimes the answer really is just inertia or fear.

4. If it's worth doing, it's worth doing well.

Almost nothing is worth doing "well." Many things can and should be done to a minimal standard of quality, because anything more is a waste of effort that can be better allocated. The exceptions aren't things that should be done "well," but things that should be done to an exceptionally high standard, probably a much higher one than you see modeled by the people around you.

This is not semantics; most people really "do everything the way they do anything" — whether that's in a low-effort or high-effort way. But the point of phoning it in on a lot of stuff is to save your energy to expend on the 1 in 1000 things that really matter … and then to actually spend it.

A recent example: I decided that I want to narrate my own audiobook for *You Can Just Do Things*. When I told a friend in the industry this and asked whether they knew any good coaches to work with, they said that hiring a coach wasn't normal or necessary, because my publisher would decide based on a sample whether I was good enough at reading to do it, and if so give me a producer to work with day-of. But it "1 in 1000" matters to me whether the audiobook is so good no one would ever think of returning it on account of the narration, so I'm not interested in the normal-shaped solution that constitutes doing it "well," I'm interested in the one that causes me to come to mind when someone asks this question. In this case, that means hiring multiple coaches and practicing for months.

5. If you like your job, you're doing something wrong.

Back when I was more alienated from my emotions, I thought about work like this: You have a budget of energy to spend, and the goal is to efficiently convert that energy into impact. You identify the activity that generates the highest impact per unit of energy, and then you spam that button for as many waking hours as you can stomach. Is it any wonder I was always burning out?

What I failed to understand is that energy is not fixed. It's dramatically affected by what you choose to do — some activities increase your energy over time, others deplete it. This feels so obvious to me now that I feel kind of stupid including it here, but I know many people (they are endemic to the Bay Area) who still labor under the false belief. Whether you enjoy what you're doing affects your ability to do it year in and year out, and dismissing this as "selfish" doesn't make it untrue. As my husband puts it, the best use of effortful motivation is to engineer your life to require less of it.

6. “All people are insane. They will do anything at any time, and God help anybody who looks for reasons.”

This line, from Kurt Vonnegut, used to neatly encapsulate the way I understood other people — which is to say, not at all. Trying to figure out why people did what they did, or imagine what they’d do next, seemed like a fool’s errand. I was judgmental about it, too: I assumed this was a problem, not with me, but with everyone else.

I didn’t fundamentally move beyond this belief until recent years, when I came into contact with the Enneagram. I imagine there are other personality typing systems that can do something similar, but the Enneagram was magical for me because it clued me into the fact that people can have very different motivational systems from my own that are still internally coherent and produce good things in the world.

People’s behavior was previously incomprehensible to me because I was imputing my own motivational system to them, trying to figure out what would cause me to act the way they did — essentially, concluding that *their* actions made no sense in light of *my* goals. But people have wildly diverse emotional hardware, which determines what’s rational for them. And my particular personality — independent, moralistic, systematic, challenge-seeking, competitive — is unusual, so I was especially poorly placed to measure others by the yardstick of myself.

This shift in perspective was dramatically good for me and for my relationships with other people. It let me see the ecosystem of psychologies as a kind of miracle — wow, it really does take all kinds — rather than a prompt for cosmic horror. And it gave me confidence that, if I invested in understanding people more deeply, they would eventually cease to be total mysteries, prone to bizarre and unreasonable behavior. As far as I can tell, this is the better part of emotional intelligence.

7. Money can’t buy happiness.

The good version of this advice is: You’ll still have to work at being happy, even if you’re rich. Money in your bank account won’t do it for you, and wealth comes with its own pathologies, like endless status-chasing. But the common gloss — that money stops mattering past some modest threshold — is basically false.

You may have heard there’s research showing money doesn’t buy happiness. That research was explosively popular precisely because it was counterintuitive — it betrayed the commonsense intuition that more money means more freedom, more options, more ability to help people you care about. Turns out it was also riddled with methodological problems. More recent work by Matthew Killingsworth (nominative determinism FTW), using large-scale studies that pinged participants about their happiness at random moments, found steady returns to well-being with

rising income, no plateau in sight. The myth persists because it serves a psychological function: It helps people without money feel better about their situation, and helps the wealthy downplay their advantages. But it's not true.

8. Intuition is fake and rationality solves everything.

I used to be horribly judgmental of people who made decisions based on intuition, as in, "I just do what feels right." And then I noticed that whenever I overrode my intuitions, I made *terrible* decisions. I hired the wrong people, started projects with the wrong people, dated the wrong people, ate food that made me feel bad, did forms of exercise I found unfun and injurious. I don't have a satisfying theory for why this is, but I know what happens: When I shift from emotional intuition to a logical story about a decision, it's much easier for my reasoning to get hijacked by plausible-sounding directives that turn out to be nonsense.

It took me a long time to learn this, and that might have something to do with my training in law and poker. Overriding intuition in favor of explicit reasoning makes sense in contexts that reward systematic rationality, where you will get separated from your money if you count on gut feelings. But most of life isn't a closed system with enumerated rules and clear outcomes. If you try to explicitly name all the factors that make someone a good friend or co-founder, you might come up with some good indicators, but your crude map will also mislead you about the territory.

This is why "wisdom is knowledge that can't be transmitted." Wise people have navigational skill, not a long list of rules.

Sign up to be notified when my book, You Can Just Do Things, is available for purchase.

The Bag-of-Documents Model for Query Understanding and Retrieval

DANIEL TUNKELANG · 19 MAY 2026 · SOURCE

Today I had the pleasure of presenting my recent work on the bag-of-documents model as part of a series of talks hosted by Trey Grainger and Doug Turnbull in the context of their AI-powered search classes.

You can find the slides below. I also encourage you to check out the bag-of-documents Github repository, the dataset and model on Hugging Face, as well as the live Amazon demo and Best Buy demo. Enjoy!

https://medium.com/media/12dd1c01b9dd78114555f2c6ed60a2fe/href